Worksheet No. 7

Student Name: ANINDITA DHAR **UID:** 25MCA20259
Branch: MCA (GENERAL) **Section/Group:** MCA- 1-A
Semester: II **Date of Performance:** 31/03/2026

Subject Name: Technical Training **Subject Code:** 25CAP-652

1.Aim/Overview of the practical:

Implementation of joins in PostgreSQL (inner join, left join, right join, self join and cross join)

2. Objectives:

Apply joins to a real-world database schema (e.g., Students, Courses, Enrollments, Departments)

3. Input/Apparatus Used:

- PostgreSQL
- pgAdmin

4. Practical / Experiment Steps

1. Write queries to list students with their enrolled courses (INNER JOIN).
2. Find students not enrolled in any course (LEFT JOIN).
3. Display all courses with or without enrolled students (RIGHT JOIN).
4. Show students with department info using SELF JOIN or multiple joins.
5. Display all possible student-course combinations (CROSS JOIN)

5.Procedure/Algorithm/Code:

i.Start the system and log in to the computer.
ii. Open PostgreSQL software.

iii. **Create four tables using the DDL command.**

```
CREATE TABLE Students (
  Student_id int,
  Student_name VARCHAR(100)
  );
```

```
CREATE TABLE Courses(
course_id int,
Course_name VARCHAR(100)
);

CREATE TABLE Enrollments(
Student_id int,
course_id int,
Course_name VARCHAR(100)
);

CREATE TABLE Departments(
Department_id INT,
Department_name VARCHAR(100),
Student_id INT
);
```

iv. Insert records into all the tables.

```
INSERT INTO  Students VALUES (1,'Anish');
INSERT INTO  Students VALUES (2,'Anil');
INSERT INTO  Students VALUES (3,'Snehasish');
INSERT INTO  Students VALUES (4,'Anindita');
INSERT INTO  Students VALUES (5,'Rani');
INSERT INTO  Students VALUES (6,'Rajni');
INSERT INTO  Students VALUES (7,'Ranibala');

SELECT * FROM Students;
```

	student_id integer	student_name character varying (100)
1	1	Anish
2	2	Anil
3	3	Snehasish
4	4	Anindita
5	5	Rani
6	6	Rajni
7	7	Ranibala

```
INSERT INTO  Courses VALUES (101,'DBMS');
INSERT INTO  Courses VALUES (102,'FRONTEND');
INSERT INTO  Courses VALUES (103,'BACKEND');
INSERT INTO  Courses VALUES (104,'COMPUTER NETWORKS');
```

INSERT INTO Courses VALUES (105,'CYBERSECURITY');
INSERT INTO Courses VALUES (106,'ADVANCED INTERNET PROGRAMMING');

SELECT * FROM Courses;

	course_id integer	course_name character varying (100)
1	101	DBMS
2	102	FRONTEND
3	103	BACKEND
4	104	COMPUTER NETWORKS
5	105	CYBERSECURITY
6	106	ADVANCED INTERNET PROGRAMMI...
7	106	ADVANCED INTERNET PROGRAMMI...

INSERT INTO Enrollments VALUES (1,101,'DBMS');
INSERT INTO Enrollments VALUES (1,102,'FRONTEND');
INSERT INTO Enrollments VALUES (2,103,'BACKEND');
INSERT INTO Enrollments VALUES (3,104,'COMPUTER NETWORKS');
INSERT INTO Enrollments VALUES (4,105,'CYBERSECURITY');
INSERT INTO Enrollments VALUES (5,103,'BACKEND');
INSERT INTO Enrollments VALUES (5,104,'COMPUTER NETWORKS');

SELECT * FROM Enrollments;

	student_id integer	course_id integer	course_name character varying (100)
1	1	101	DBMS
2	1	102	FRONTEND
3	2	103	BACKEND
4	3	104	COMPUTER NETWORKS
5	4	105	CYBERSECURITY
6	5	104	COMPUTER NETWORKS
7	5	103	BACKEND

INSERT INTO Departments VALUES (1,'MCA',1);
INSERT INTO Departments VALUES (11,'MCA',2);
INSERT INTO Departments VALUES (12,'ENGINEERING',3);
INSERT INTO Departments VALUES (13,'ENGINEERING',4);
INSERT INTO Departments VALUES (12,'BCA',5);
INSERT INTO Departments VALUES (12,'BCA',6);
INSERT INTO Departments VALUES (12,'BCA',7);

	department_id integer	department_name character varying (100)	student_id integer
1	1	MCA	1
2	11	MCA	2
3	12	ENGINEERING	3
4	13	ENGINEERING	4
5	12	BCA	5
6	12	BCA	6
7	12	BCA	7

v. Write queries to list students with their enrolled courses (INNER JOIN).

```
SELECT s.Student_id, s.Student_name, e.course_id, e.Course_name
FROM Students AS s
JOIN Enrollments AS e
ON s.Student_id = e.Student_id
ORDER BY Student_id ASC;
```

	student_id integer	student_name character varying (100)	course_id integer	course_name character varying (100)
1	1	Anish	101	DBMS
2	1	Anish	102	FRONTEND
3	2	Anil	103	BACKEND
4	3	Snehasish	104	COMPUTER NETWORKS
5	4	Anindita	105	CYBERSECURITY
6	5	Rani	104	COMPUTER NETWORKS
7	5	Rani	103	BACKEND

vi. Find students not enrolled in any course (LEFT JOIN).

```
SELECT s.Student_id, s.Student_name
FROM Students AS s
LEFT JOIN Enrollments as e
ON s.Student_id = e.Student_id
WHERE e.Student_id IS NULL
ORDER BY s.Student_id ASC;
```

	student_id integer	student_name character varying (100)
1	6	Rajni
2	7	Ranibala

vii. Display all courses with or without enrolled students (RIGHT JOIN).

```
SELECT DISTINCT(c.course_id),e.Course_name
FROM Enrollments AS e
RIGHT JOIN Courses AS c
ON e.course_id=c.course_id
ORDER BY course_id ASC;
```

	course_id integer	course_name character varying (100)
1	101	DBMS
2	102	FRONTEND
3	103	BACKEND
4	104	COMPUTER NETWORKS
5	105	CYBERSECURITY
6	106	[null]

viii.Show students with department info using SELF JOIN or multiple joins.

```
SELECT s.Student_id, s.Student_name,
d.Department_id,d.Department_name,e.course_id,e.Course_name
FROM Students AS s
LEFT JOIN Departments AS d ON s.Student_id = d.Student_id
LEFT JOIN Enrollments AS e ON  s.Student_id = e.Student_id
ORDER BY s.Student_id ASC;
```

	student_id integer	student_name character varying (100)	department_id integer	department_name character varying (100)	course_id integer	course_name character varying (100)
1	1	Anish	1	MCA	101	DBMS
2	1	Anish	1	MCA	102	FRONTEND
3	2	Anil	11	MCA	103	BACKEND
4	3	Snehasish	12	ENGINEERING	104	COMPUTER NETWORKS
5	4	Anindita	13	ENGINEERING	105	CYBERSECURITY
6	5	Rani	12	BCA	104	COMPUTER NETWORKS
7	5	Rani	12	BCA	103	BACKEND
8	6	Rajni	12	BCA	[null]	[null]
9	7	Ranibala	12	BCA	[null]	[null]

ix. Display all possible student-course combinations (CROSS JOIN).

```
SELECT s.Student_id,s.Student_name,c.course_id,c.Course_name,
FROM Students AS s
CROSS JOIN Courses AS c
ORDER BY s.Student_id ASC;
```

	student_id integer	student_name character varying (100)	course_id integer	course_name character varying (100)
1	1	Anish	101	DBMS
2	1	Anish	102	FRONTEND
3	1	Anish	103	BACKEND
4	1	Anish	104	COMPUTER NETWORKS
5	1	Anish	105	CYBERSECURITY
6	1	Anish	106	ADVANCED INTERNET PROGRAMMI...
7	1	Anish	106	ADVANCED INTERNET PROGRAMMI...
8	2	Anil	101	DBMS
9	2	Anil	102	FRONTEND
10	2	Anil	103	BACKEND
11	2	Anil	104	COMPUTER NETWORKS
12	2	Anil	105	CYBERSECURITY
13	2	Anil	106	ADVANCED INTERNET PROGRAMMI...
14	2	Anil	106	ADVANCED INTERNET PROGRAMMI...
15	3	Snehasish	101	DBMS
16	3	Snehasish	102	FRONTEND
17	3	Snehasish	103	BACKEND
18	3	Snehasish	104	COMPUTER NETWORKS
19	3	Snehasish	105	CYBERSECURITY
20	3	Snehasish	106	ADVANCED INTERNET PROGRAMMI...
21	3	Snehasish	106	ADVANCED INTERNET PROGRAMMI...
22	4	Anindita	101	DBMS
23	4	Anindita	102	FRONTEND
24	4	Anindita	103	BACKEND
25	4	Anindita	104	COMPUTER NETWORKS
26	4	Anindita	105	CYBERSECURITY
27	4	Anindita	106	ADVANCED INTERNET PROGRAMMI...
28	4	Anindita	106	ADVANCED INTERNET PROGRAMMI...
29	5	Rani	101	DBMS
30	5	Rani	102	FRONTEND
31	5	Rani	103	BACKEND
32	5	Rani	104	COMPUTER NETWORKS
33	5	Rani	105	CYBERSECURITY
34	5	Rani	106	ADVANCED INTERNET PROGRAMMI...
35	5	Rani	106	ADVANCED INTERNET PROGRAMMI...
36	6	Rajni	101	DBMS
37	6	Rajni	102	FRONTEND
38	6	Rajni	103	BACKEND
39	6	Rajni	104	COMPUTER NETWORKS
40	6	Rajni	105	CYBERSECURITY
41	6	Rajni	106	ADVANCED INTERNET PROGRAMMI...
42	6	Rajni	106	ADVANCED INTERNET PROGRAMMI...
43	7	Ranibala	101	DBMS
44	7	Ranibala	102	FRONTEND
45	7	Ranibala	103	BACKEND
46	7	Ranibala	104	COMPUTER NETWORKS
47	7	Ranibala	105	CYBERSECURITY
48	7	Ranibala	106	ADVANCED INTERNET PROGRAMMI...
49	7	Ranibala	106	ADVANCED INTERNET PROGRAMMI...

7. Learning Outcomes

- Executing SQL queries with JOINS eg. LEFT JOIN, RIGHT JOIN ,SELF JOIN and CROSS JOIN to carry out multiple functionalities.
- Implementing multiple joins to carry out a wide array of functionalities.